

TODO:

– Ausfüllrätsel

– Entscheidbarkeit

– Halteproblem

– Liste der NP-Vollständigen Probleme

– TM Berechnungsgeschichte

– extras

– CNF konstruieren

– Minimalautomat

– Mengenoperationen

– DEA/NEA/TM/NTM

– Beispiele:

– Ist Sprache x regulär? → Pumping Lemma

– Ist Sprache x kontextfrei? → Pumping Lemma

– Kann eine NTM in polynomieller Zeit entscheiden, ob ein Ausfüllrätsel eine Lösung hat? → Satz von Rice / P Verifizierer / Reduktion

– Ist Grammatik eindeutig? (FS_2026)5))

– Parse-tree finden (FS_2022)5))

– Prüfungsaufgaben:

– HS_2021)4)

– FS_2022)4)

Begriff	Bedeutung
Regulär	Es gibt einen DEA A, der L akzeptiert, also $L(A) = L$
Kontextfrei	L kann nur von einem ND PDA erkannt werden

VORGEHEN ZU AUFGABENTYPEN

Ist Sprache regulär?	Ja: DEA / NEA / RegEx Nein: Pumping Lemma
Ist Sprache kontextfrei?	Ja: Stackautomat / CNF / BNF Nein: Pumping Lemma (CFL)
Turing-Maschinen	TODO: Berechnungsgeschichte
Ist eine Sprache Turing-Vollständig?	TODO: Ja: TM-Simulation Nein: Halteproblem
Kann Problem von einem Computer gelöst werden?	TODO: Reduktion auf Halteproblem

Kann ein Programm entscheiden, ob zulässige Inputs eine Eigenschaft erfüllen?	TODO: Satz von Rice
---	-------------------------------

Kann eine nichtdeterministische Maschine ... entscheiden?	TODO: NP-Verifizierer
---	---------------------------------

Warum kann ein Problem nicht gelöst werden?	Reduktion auf NP-Vollständiges Problem
---	--

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

$A = (Q, \Sigma, \delta, q, F)$

– Zustände: $Q = \{q_0, q_1, \dots, q_n\}$

– Alphabet: Σ

– Übergangsfunktion: $\delta : Q \times \Sigma \rightarrow Q$

– Startzustand: $q \in Q$

– Akzeptierzustände: $F \subset Q$

Q

F

q ₀	q ₂	q ₁
q ₁	q ₀	q ₁
q ₂	q ₁	q ₂

δ

PUMPING LEMMA (DEA)

Um zu beweisen, dass eine Sprache **nicht** regulär ist

Vorgehen:

- 1) Annahme: L ist regulär
- 2) Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- 3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition **muss** N verwenden!
- 4) Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz, |xy| \leq N, |y| > 0$
- 5) Auswirkung des Pumpens
 - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 1 : $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- 1) Annahme: L ist regulär
- 2) $\exists N \in \mathbb{N}$, Pumping Length
- 3) $w = 0^N 1^N$
- 4) Unterteilung $w = xyz$
- 5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt
- 6) $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

Beispiel 2

TODO:

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

$$A = (Q, \Sigma, \delta, q, F)$$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta : Q \times \Sigma \rightarrow P(Q)$
- Startzustand: $q \in Q$
- Akzeptierzustände: $F \subset Q$

TODO:
Thompson-NEA

TODO:
RegEx Mengenoperationen

KONTEXTFREIE SPRACHEN (CFL)

$$G = (V, \Sigma, R, S)$$

- V: Variablen
- Σ : Terminalsymbole (Alphabet)
- R: Regeln der Form $A \rightarrow x_1 x_2 \dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$
- S: Startvariable

TODO:
parse-tree

PUMPING LEMMA (CFL)

Um zu beweisen, dass eine Sprache **nicht** kontextfrei ist

Voraussetzungen:

- 1) $|vxy| > 0$
- 2) $|vxy| \leq N$
- 3) $uv^kxy^kz \in L \forall k \in \mathbb{N}$

Beispiel 3

TODO:

CHOMSKY-NORMALFORM (CNF)

- S kommt auf der rechten Seite nicht vor
- Jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$
- $S \rightarrow \epsilon$ ist erlaubt

Beispiel 4

TODO:

STACKAUTOMAT (PDA)

$$P = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- 1) Q: Zustände
- 2) Σ : Eingabe-Alphabet
- 3) Γ : Stack-Alphabet
- 4) $\delta : Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma \times \Gamma)$
- 5) $q_0 \in Q$: Startzustand
- 6) $F \subset Q$: Akzeptierzustände

- a vom Input
- b vom Stack entfernen (Bedingung)

c auf den Stack

TURING MASCHINEN

(deterministische) Turing-Maschine

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

- Q: Zustände
- Σ : Alphabet
- Γ : Bandalphabet, $\sqcup \in \Gamma \setminus \Sigma$
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- $q_0 \in Q$: Startzustand
- $q_{\text{accept}} \in Q$: Akzeptierzustand
- $q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$

- Übergang möglich, wenn a unter dem Schreib- / Lese-Kopf
- Aktuelles Feld auf dem Band wird mit b überschrieben
- Kopfbewegung: L links, R rechts

TODO:
berechnungsgeschichte

TODO:
varianten

ENTSCHEIDBARKEIT

TODO:
Entscheidbare Probleme

SATZ VON RICE

Ist P eine nichttriviale Eigenschaft Turing-erkennbarer Sprachen, dann ist P_{TM} nicht entscheidbar.

Eine Eigenschaft P Turing-erkennbarer Sprachen heisst **nichttrivial**, wenn es zwei Turing-Maschinen M_1 und M_2 gibt, wobei M_1 die Eigenschaft hat und M_2 nicht.

LÖSUNGSANLEITUNG EINER PRÜFUNGSFRAGE:

- Nichttriviale Eigenschaft P aufschreiben
- Die beiden Sprachen $L(M_1)$ und $L(M_2)$ bilden (meistens kann man die leere Menge oder Σ^* als eine dieser Sprachen verwenden)
- Gibt es ein Programm, welches beide Sprachen erkennen kann? Sind beide Sprachen Turing erkennbar?
- Dann besagt der Satz von Rice, dass die Sprache nicht entscheidbar ist

TODO:

Beispiel

NP

TODO:

POLYNOMIELLER VERIFIZIERER

TODO:

NURIKABE

Entscheidbar? Ja: $n = uv = \text{Anzahl Felder}$. Alle Belegungen des Spielfeldes mit schwarzen Feldern können überprüft werden, ob sie die Regeln einhalten.

Zertifikat: Liste der schwarzen Felder

Vorgehen:

Schritt	Aufwand
1 Für jedes Zahlfeld ($O(n)$) verwendet man einen Markieralgorithmus, der alle zum Gebiet dieses Zahlfeldes gehörenden weissen Felder bestimmt ($O(n)$ Durchgänge mit Aufwand $O(n)$).	$O(n^3)$
2 Trifft dieser Algorithmus auf ein weiteres Zahlfeld, ist der erste Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$O(n)$
3 Weicht die Zahl der gefundenen weissen Felder vom Inhalt des Zahlfeldes ab, ist der zweite Teil von Regel 1 verletzt ($\rightarrow q_{\text{reject}}$).	$n \cdot O(n)$
4 Ebenfalls mit einem Markieralgorithmus wird überprüft, ob das schwarze Gebiet zusammenhängend ist.	$O(n^2)$
5 Für jedes schwarze Feld wird überprüft, ob es Teil eines 2×2 -Tümpels ist.	$O(n)$
Total	$O(n^3)$

REDUKTION

TODO:

NP-VOLLSTÄNDIG

Reduktion auf NP-Vollständiges Problem

Zu beachten (Punkteverteilung):

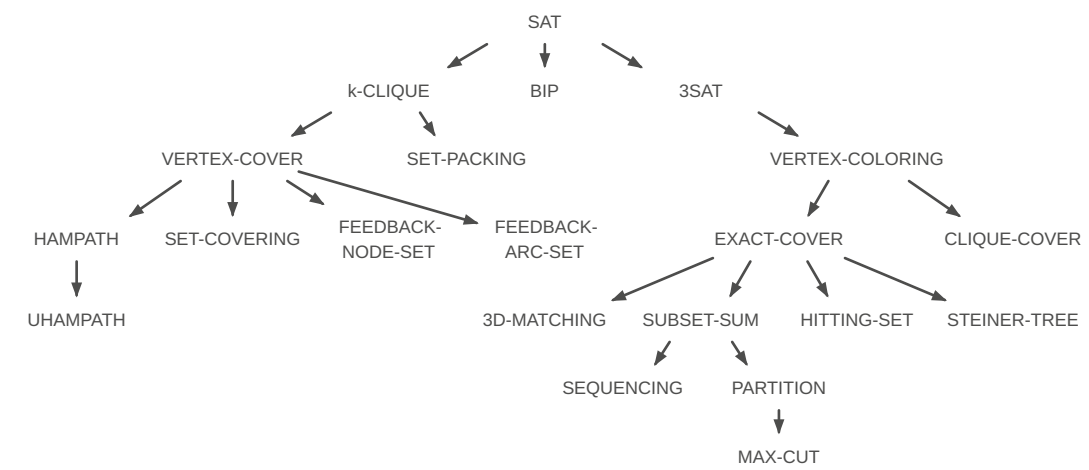
- Lösungsprinzip der Reduktion erwähnen (1P)
- Auswahl eines geeigneten Vergleichsproblems (1P)
- Reduktionsabbildung (*P)
- Schlussfolgerung: NP-Vollständig und somit nicht effizient lösbar (1P)

TODO:

AutoSpr | FS26

Seite 1

LOGIK



DIFFERENZMENGE $L(A_1) \setminus L(A_2)$

A_2

$A_1 \setminus A_2$

A_1

$A_1 \times A_2$

MENGEN
SCHNITTMENGE $L(A_1) \cap L(A_2)$

A_2

$A_1 \cap A_2$

A_1

$A_1 \times A_2$

SYMMETRISCHE DIFFERENZ $L(A_1) \triangle L(A_2)$

A_2

$A_1 \triangle A_2$

A_1

$A_1 \times A_2$

VEREINIGUNGSMENGE $L(A_1) \cup L(A_2)$

A_2

$A_1 \cup A_2$

A_1

$A_1 \times A_2$

AutoSpr | FS26

Seite 3